

Kriptovaluták elméleti háttere

Ladóczki Bence

docens

Távközlési és Mesterséges Intelligencia Tanszék (TMIT)

Budapesti Műszaki és Gazdaságtudományi Egyetem

Villamosmérnöki és Informatikai Kar (VIK)

ladoczki.bence@vik.bme.hu

2026. szeptember 17.

Tartalomjegyzék

1	ECDSA a gyakorlatban	2
1.1	ECDSA – elliptikus görbék feletti DSA	2
1.2	A secp256k1 görbe	2
1.3	Digitális aláírások a gyakorlatban: blokkláncok összehasonlítása	3
2	Threshold kriptográfia	4
2.1	Motiváció és alkalmazások	4
2.2	Threshold vs. multisig	4
2.3	A (t, n) threshold séma	4
2.4	Threshold ECDSA kihívásai	5
2.5	Presignature: gyors online aláírás	5
2.6	FROST: körre optimalizált Schnorr threshold aláírás	5
2.6.1	Miért fontos, mi a cél?	5
2.6.2	Kulcsgenerálás (KeyGen)	6
2.6.3	Előfeldolgozás és aláírás (Sign)	7
2.6.4	Mit érünk el, milyen felhasználása van?	9
3	BLS aláírás	9
3.1	Bilineáris párosítás	10
3.2	Weil párosítás	10
3.2.1	Történeti háttér	10
3.2.2	Matematikai alapok	12
3.2.3	A Weil-párosítás definíciója	12
3.2.4	Miller-algoritmus: implementáció	13
3.2.5	Kriptográfiai alkalmazások	13
3.3	BLS kulcsgenerálás	15
3.4	BLS aláírás generálás	15
3.5	BLS aláírás ellenőrzés	15
3.6	Aláírás aggregáció	16
3.7	BLS hátrányai	16

A secp256k1 a Bitcoin 2009-es megjelenésekor szokatlan választás volt – a legtöbb rendszer a NIST által ajánlott secp256r1 (más néven P-256) görbét használta. A secp256k1 választásának valószínű okai:

1. Egyszerű struktúra: Az $a = 0$ paraméter miatt a görbe felépítése átlátható, nincs benne “rejtett” konstans. A NIST görbék paramétereinek eredete nem teljesen transzparens (a kriptográfiai közösség egy része gyanakodott, hogy az NSA backdoort épített beléjük).
2. Hatékonyság: A Koblitz-típusú görbéken a pontösszeadás és skalárszorítás gyorsabb mint az általános Weierstrass-görbéken.
3. Endomorfizmus gyorsítás: A secp256k1 rendelkezik egy hatékony endomorfizmussal (λ -endomorfizmus, a GLV-módszer [28] alkalmazható), amely a skalársorzást kb. 30–40%-kal gyorsítja.
4. Kofaktor $h = 1$: A teljes görbe egyetlen ciklikus csoport, nincs kis rendű alcsoport – ez egyszerűsíti az implementációt és kivéd bizonyos támadásokat.

Biztonsági szint: A 256 bites görbe kb. 128 bites biztonsági szintet nyújt (a legjobb ismert támadás, a Pollard- ρ , $O(\sqrt{n}) \approx 2^{128}$ lépést igényel). Ez ma biztonságosnak tekinthető, mert 2^{128} művelet elvégzése a jelenlegi (és a belátható jövőbeli) számítási kapacitással is kivitelezhetetlen: még ha egy támadó másodpercenként 10^{18} ($\approx 2^{60}$) műveletre képes gépet üzemeltetne, akkor is nagyságrendileg az univerzum életkorát meghaladó ideig tartana a kulcskereset végigpróbálni.

1.3 Digitális aláírások a gyakorlatban: blokkláncok összehasonlítása

Az alábbi táblázat összefoglalja, hogy a legismertebb blokklánc hálózatok milyen digitális aláírás algoritmust használnak:

Blokklánc	Aláírás algoritmus	Görbe / paraméterek
Bitcoin	ECDSA + Schnorr (Taproot óta)	secp256k1
Ethereum	ECDSA	secp256k1
Solana	EdDSA	Ed25519 (Curve25519)
Cardano	EdDSA	Ed25519
Polkadot	Schnorr (sr25519) + EdDSA	Ristretto25519 / Ed25519
Ripple (XRP)	ECDSA + EdDSA	secp256k1 / Ed25519
Avalanche	ECDSA	secp256k1
Cosmos	ECDSA + EdDSA	secp256k1 / Ed25519
Tezos	EdDSA + ECDSA + BLS	Ed25519 / secp256k1 / BLS12-381
Monero	Borromean / CLSAG	Ed25519

Érdemes megfigyelni a tendenciát: a korábbi blokkláncok (Bitcoin, Ethereum) a secp256k1 feletti ECDSA-t választották, míg az újabb generáció (Solana, Cardano, Polkadot) inkább az Ed25519 görbét és az EdDSA (Edwards-curve Digital Signature Algorithm) aláírást preferálja. Ennek oka, hogy az EdDSA determinisztikus (nincs nonce-újrafelhasználási kockázat), egyszerűbb az implementáció, és a Curve25519/Ed25519 görbét kifejezetten úgy tervezték, hogy ellenálló legyen az implementációs hibákból adódó oldalsó csatornás támadásokkal szemben.

2 Threshold kriptográfia

A threshold (küszöb) kriptográfia olyan protokollokat jelent, amelyekben egy kriptográfiai művelet (pl. aláírás generálás vagy visszafejtés) nem egyetlen fél titkos kulcsával történik, hanem n fél között megosztott kulcs-részletekkel, amelyekből legalább $t + 1$ fél együttműködése szükséges. A rendszer biztonsága garantálja, hogy t vagy kevesebb kompromittált fél semmilyen információt nem szerez a teljes kulcsról, és nem tud érvényes aláírást generálni.

2.1 Motiváció és alkalmazások

A threshold kriptográfia a következő gyakorlati problémákat oldja meg:

- Többszörös aláírók: bizalmas dokumentumok aláírásához vagy titkosított adatok visszafejtéséhez kvórum (többségi hozzájárulás) szükséges.
- Kulcsvédelem: a privát kulcs egyetlen ponton való tárolása (single point of failure) helyett több eszköz vagy felhasználó között oszlik meg.
- Ellenállóság a támadásokkal szemben: a kulcs megszerzéséhez a támadónak egyszerre több eszközt kell feltörnie.

A terület intenzív kutatási fázison ment át a 90-es és 2000-es években, majd a kriptovaluták megjelenésével újra fellendült: a Wallet-as-a-Service (WaaS) cégek (Fireblocks [67], Dfns, Coinbase, Curv [68]) aktívan alkalmazzák kulcsvédelemre, mert a Bitcoin/Ethereum privát kulcs ellopása azonnali, visszafordíthatatlan pénzügyi veszteséget jelent.

2.2 Threshold vs. multisig

A Bitcoin natív multisig megoldásával szemben a threshold kriptográfia számos előnnyel rendelkezik:

- A multisig a blokkláncon látható (az összes publikus kulcs megjelenik a szkriptben), míg a threshold aláírás egyetlen normál aláírásnak tűnik – jobb anonimitás.
- A multisig hozzáférési struktúrája merev (pl. 2-of-3), míg a threshold protokollok rugalmasabb struktúrákat támogatnak.
- A threshold megoldás skálázhatóbb: az aláírás mérete független a résztvevők számától.

2.3 A (t, n) threshold séma

Egy (t, n) threshold sémában n szerver (fél) osztozik a privát kulcsra. Minden szerver egy kulcs-részletet (share) tárol, és legalább $t + 1$ szerver együttműködése szükséges az aláírás generálásához. A biztonsági garanciák:

- Kulcsbiztonság: t vagy kevesebb kompromittált szerver semmit sem tud meg a privát kulcsról (information-theoretic security, Shamir titkos megosztás [65] alapján).
- Aláírási védelem: t vagy kevesebb kompromittált szerver nem tud érvényes aláírást generálni.

A Shamir-féle titkos megosztás a Lagrange-interpoláción alapul: a titok s egy t -edfokú $f(x)$ polinom konstans tagja ($f(0) = s$), és minden fél megkapja $f(i)$ -t (egy pontot a polinomon). Bármely $t + 1$ pont egyértelműen meghatározza a polinomot (és így s -t), de t vagy kevesebb pont semmilyen információt nem ad s -ről.

2.4 Threshold ECDSA kihívásai

Az ECDSA a leggyakrabban használt aláírási séma a blokkláncokban (Bitcoin, Ethereum), de a threshold megvalósítása jóval nehezebb, mint a Schnorr aláírásoké. A technikai probléma az aláírás $s = k^{-1} \cdot (H(m) + x \cdot r) \bmod q$ képletében rejlik: a k^{-1} kiszámítása k ismerete nélkül – amikor k maga is megosztott a felek között – nem triviális, mert a moduláris inverz nem lineáris művelet.

Az első áttörést MacKenzie és Reiter [69] érték el egy két-feles megoldással, amely Paillier-titkosítást (additíven homomorf titkosítás) alkalmaz: az egyik fél titkosított formában kapja meg a másik fél részértékeit, és a homomorf tulajdonság lehetővé teszi az aritmetikai műveletek végrehajtását anélkül, hogy a titkos értékek felfedetlenül maradnának. A modern protokollok [38, 66] ezt általánosítják n félre.

2.5 Presignature: gyors online aláírás

A modern threshold ECDSA sémák két fázisra bontják az aláírást:

1. Offline (presignature) fázis: a felek interaktív, számításigényes protokollt futtatnak, amely előállítja a megosztott nonce-t és a presignature állapotot. Ez a fázis az üzenet ismerete nélkül végrehajtható. A *presignature állapot* nem más, mint az aláírás összes olyan része, amely nem függ az m üzenettől, előre kiszámítva és a felek között megosztott (titkos) formában eltárolva: az $s = k^{-1}(H(m) + x \cdot r) \bmod q$ képletben ez az $r = (kG)_x$ komponens és a k^{-1} megosztott értéke (illetve a $k^{-1}x$ szorzat megosztása). Miután ezek megvannak, az online fázisban már csak a $H(m)$ -től függő tagot kell hozzáadni – ez az egyetlen olcsó lépés, amit nem lehetett előre elvégezni.
2. Online fázis: amikor az aláírandó üzenet megérkezik, a presignature felhasználásával egyetlen, nem-interaktív (vagy minimálisan interaktív) lépésben generálható az aláírás.

Ez a felbontás lehetővé teszi, hogy az online késleltetés minimális legyen (milliszekundum nagyságrendű), míg a költséges számítás az offline fázisban, előre elvégezhető.

2.6 FROST: körre optimalizált Schnorr threshold aláírás

A Schnorr aláírások threshold változatának egyik legfontosabb modern konstrukciója a **FROST** (Flexible Round-Optimized Schnorr Threshold), amelyet Chelsea Komlo (University of Waterloo, Zcash Foundation) és Ian Goldberg (University of Waterloo) publikált 2020-ban [33].

2.6.1 Miért fontos, mi a cél?

Az ECDSA-val ellentétben (lásd az előző alfejezetet) a Schnorr aláírás lineáris szerkezetű, ezért a threshold megvalósítása elvben egyszerűbb: az egyes felek részaláírásai egyszerűen összeadhatók. A gyakorlati nehézség máshol van: a threshold aláírás előállításához a

feleknek hálózati *körökben* (round) kell kommunikálniuk egymással, és minél több a kör, annál lassabb a protokoll – különösen, ha a felek korlátozott hálózati kapcsolattal rendelkező eszközökön (pl. hardveres kulcstárolók) futnak, vagy megbízhatatlan hálózaton keresztül egyeztetnek. A korábbi, párosítás nélküli Schnorr threshold sémák vagy legalább három kört igényeltek, vagy – ha kevesebb kört használtak – korlátozniuk kellett az egyidejű (konkurrens) aláírások számát, hogy védekezzenek egy ismert hamisítási támadás (a Drijvers-támadás) ellen [33].

A FROST célja, hogy ezt a hálózati többletterhelést minimálisra csökkentse: az aláírás egyetlen körben elvégezhető egy előfeldolgozási (preprocessing) fázis után, *anélkül*, hogy korlátozni kellene a párhuzamos aláírások számát. A Drijvers-támadás lényege, hogy ha a támadó *sok* aláírási munkamenetet indíthat egyszerre és látja a többiek nonce-commitmentjeit, mielőtt a sajátját véglegesítené, akkor a saját hozzájárulását utólag úgy állíthatja be, hogy a több munkamenetből származó kihívások (challenge-ek) egy általa megcélzott értékre álljanak össze. Ezzel a párhuzamos aláírások közötti algebrai összefüggést (egy Wagner-féle általánosított születésnapi támadást) kihasználva a küszöbszámnál kevesebb féllel is hamis csoport-aláírást tud kovácsolni egy soha alá nem írt üzenetre. Cserébe a FROST feladja a *robustusságot*: ha egy résztvevő hibásan viselkedik, a protokoll nem fejeződik be automatikusan, hanem megszakad (abort), a hibás felet azonosítják és kizárják, majd a protokollt újrafuttatják. A legtöbb valós alkalmazásban ez elfogadható kompromisszum, és cserébe a FROST akkor is biztonságos marad, ha a támadó a küszöbnél (t) kevesebb felet irányít – ez jobb, mint a robustus sémák $t \leq n/2$ korlátja [33].

2.6.2 Kulcsgenerálás (KeyGen)

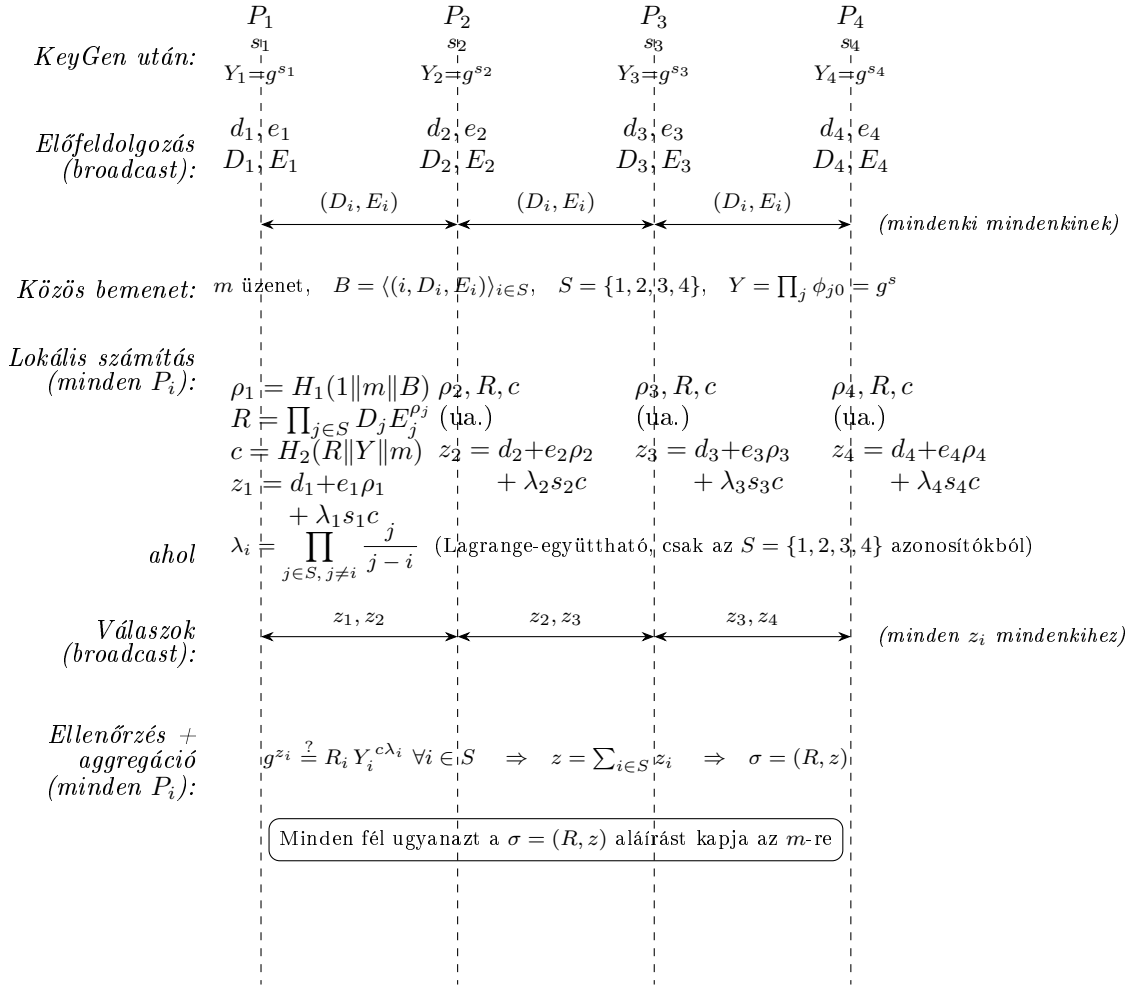
A FROST a Pedersen-féle elosztott kulcsgenerálásra (DKG, *distributed key generation*) épül, tehát *nincs* szükség megbízható központi osztóra (dealer). A jelölésben legyen $\mathbb{G}$ egy q prímmrendű ciklikus csoport g generátorral (a diszkrét logaritmus $\mathbb{G}$ -ben nehéz), n a résztvevők száma és t a küszöb (az aláíráshoz szükséges felek minimális száma). A résztvevőket $P_1, \dots, P_n$ jelöli, azonosítójuk pedig az $i, \ell \in \{1, \dots, n\}$ index, amelyet egyben a polinom kiértékelési pontjaként is használunk. Minden P_i résztvevő választ egy $t - 1$ -edfokú polinomot,

$$f_i(x) = \sum_{j=0}^{t-1} a_{ij} x^j, \quad a_{ij} \xleftarrow{\$} \mathbb{Z}_q,$$

ahol az a_{ij} együtthatókat véletlenszerűen (a $\xleftarrow{\$}$ jelöli az egyenletes véletlen választást) sorsolja $\mathbb{Z}_q$ -ból, és $a_{i0} = f_i(0)$ lesz P_i titkos hozzájárulása a közös kulcshoz. és nyilvános commitmentként közzéteszi a $\phi_{ij} = g^{a_{ij}}$ értékeket (tehát ϕ_{ij} az a_{ij} együttható „elrejtett”, diszkrét logaritmus mögé zárt formája: a_{ij} -t nem fedi fel, de lehetővé teszi az ellenőrzést). A „rogue-key” támadások ellen minden fél Schnorr-aláírással bizonyítja, hogy ismeri a saját a_{i0} titkát. Ezután minden P_i biztonságos csatornán elküldi a $(\ell, f_i(\ell))$ részt minden más P_ℓ félnek, aki ellenőrzi a kapott részt:

$$g^{f_\ell(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} \phi_{\ell k}^{i^k \bmod q}.$$

Itt $f_\ell(i)$ a P_ℓ által P_i -nek küldött részérték (a P_ℓ polinomjának értéke a P_i azonosítóját jelölő i pontban), $\phi_{\ell k} = g^{a_{\ell k}}$ pedig P_ℓ közzétett commitmentjei, $k = 0, \dots, t - 1$. Az



1. ábra: Egy FROST aláírás futása négy egyenrangú féllel ($S = \{1, 2, 3, 4\}$). Az aláírók előre, egymás közt broadcasttal megosztják a (D_i, E_i) nonce-commitmenteket; az m üzenet és a B lista minden fél előtt ismert; ezután mindenki lokálisan kiszámolja a közös R -t, a c kihívást és a saját z_i válaszát, amelyet broadcasttal szétküld; végül minden fél maga ellenőrzi a válaszokat a $g^{z_i} \stackrel{?}{=} R_i Y_i^{c \lambda_i}$ egyenlettel, összegzi őket ($z = \sum z_i$), és így mindenki ugyanazt a $\sigma = (R, z)$ csoport-aláírást kapja.

egyenlet azt igazolja, hogy a titokban kapott $f_\ell(i)$ tényleg a nyilvánosan kihirdetett polinomon fekszik: a jobb oldal a kitevőben épp a $\sum_{k=0}^{t-1} a_{\ell k} i^k = f_\ell(i)$ összeget építi fel (a $g^{a_{\ell k}}$ hatványokból, i^k kitevővel), így ha a küldő becsapná P_i -t, az egyenlőség nem állna fenn. Végül minden fél kiszámítja a hosszú élettartamú titkos kulcs-részletét, $s_i = \sum_{\ell=1}^n f_\ell(i)$, a hozzá tartozó nyilvános ellenőrző részt $Y_i = g^{s_i}$, valamint a csoport közös nyilvános kulcsát $Y = \prod_{j=1}^n \phi_{j0}$.

2.6.3 Előfeldolgozás és aláírás (Sign)

A FROST kulcs ötlete, hogy a nonce-okat *nem* egy drága DKG-vel állítja elő minden aláíráskor, hanem előre, az üzenet ismerete nélkül. Az előfeldolgozási fázisban minden P_i generál egy adag egyszer használatos nonce-párt (d_{ij}, e_{ij}) és közzéteszi a hozzájuk tartozó commitmenteket $(D_{ij}, E_{ij}) = (g^{d_{ij}}, g^{e_{ij}})$. Itt a második index, j , sorszámozza a P_i fél előre legyártott nonce-párjait ($j = 1, 2, \dots$); egy adott aláírási művelethez minden fél pontosan

egy párt választ ki a saját készletéből. A jelölés egyszerűsítése végett, amikor egy konkrét aláírásról beszélünk, a j sorszámot elhagyjuk, és a kiválasztott párt röviden (d_i, e_i) -nek, illetve (D_i, E_i) -nek írjuk – ez tehát ugyanaz, mint a P_i által épp erre az aláírásra használt (d_{ij}, e_{ij}) , (D_{ij}, E_{ij}) .

Fontos, hogy ez *nem* jelent nonce-újrafelhasználást (nonce reuse): az előfeldolgozás nem egyetlen nonce-ot gyárt le előre, hanem egy *adag különböző*, egyenként csak egyszer felhasználható nonce-párt (ezeket sorszámozza a j index), és minden párt pontosan egy aláíráshoz szabad felhasználni. A protokoll ezért elő is írja, hogy a felhasznált (d_i, e_i) nonce-ot a fél az aláírás után azonnal, biztonságosan törli. Ha egy nonce-ot mégis kétszer használnának ugyanarra a titkos részre, az ugyanaz a katasztrofális hiba, mint az egyfelhasználós Schnorr- vagy ECDSA-aláírásoknál: két, azonos (d_i, e_i) -vel készült z_i válaszból a $\lambda_i s_i c$ tagok kiejthetők, és a támadó visszafejtheti a fél hosszú távú titkos részét, s_i -t. A biztonság tehát nem az előre generáláson múlik, hanem azon, hogy minden nonce szigorúan egyszer használatos.

Aláíráskor a $t \leq \alpha \leq n$ kiválasztott aláíró (S halmaz, α a tényleges aláírók száma) a következőket számolja az m aláírandó üzenetre. Először minden fél kiszámítja a *kötő értékeket* (binding values), amelyek az üzenethez, az aláírók halmazához és a commitmentekhez kötik a részaláírást – épp ez akadályozza meg a Drijvers-támadást:

$$\rho_i = H_1(i \parallel m \parallel B), \quad B = \langle (i, D_i, E_i) \rangle_{i \in S},$$

ahol B az aláírók azonosítóit és nyilvános commitmentjeit tartalmazó rendezett lista. Ebből előáll a csoport-commitment és a Schnorr-kihívás (challenge):

$$R = \prod_{i \in S} D_i \cdot (E_i)^{\rho_i}, \quad c = H_2(R \parallel Y \parallel m).$$

Itt Y a csoport közös nyilvános kulcsa, amelyet nem a signing során, hanem már a kulcsgenerálásban rögzítünk: $Y = \prod_{j=1}^n \phi_{j0} = g^{\sum_j a_{j0}} = g^s$. Mivel a $\phi_{j0} = g^{a_{j0}}$ értékeket minden fél nyilvánosan közzétette a KeyGen alatt, ezt a szorzatot minden fél (sőt bármely külső ellenőrző is) önállóan, azonos eredménnyel kiszámítja – nincs szükség hozzá titkos adatra vagy koordinációra. (Ne keverjük össze a résztvevők egyéni $Y_i = g^{s_i}$ ellenőrző részeivel: a challenge-ben a közös Y szerepel.) Minden fél kiszámítja a saját válaszát a hosszú élettartamú s_i titok és a λ_i Lagrange-együttható felhasználásával:

$$z_i = d_i + (e_i \cdot \rho_i) + \lambda_i \cdot s_i \cdot c.$$

Itt H_1, H_2 olyan hash-függvények, amelyek kimenete a $\mathbb{Z}_q^*$ halmazba esik, λ_i pedig az i fél Lagrange-együtthatója az aktuális S aláíró-halmaz felett,

$$\lambda_i = \prod_{j \in S, j \neq i} \frac{j}{j - i},$$

amely a Shamir-féle titokmegosztásból ismert interpolációs súly: ez alakítja át a felek additív részeit a közös titok helyes rekonstrukciójává (csak az azonosítóktól függ, a titkos részekről nem). A d_i, e_i az előfeldolgozásból származó egyszer használatos nonce-ok, ρ_i a kötő érték, c pedig a közös challenge. Minden fél ellenőrzi a válaszokat a $g^{z_i} \stackrel{?}{=} R_i \cdot Y_i^{c \cdot \lambda_i}$ egyenlettel (ahol $R_i = D_i \cdot (E_i)^{\rho_i}$ az i fél commitment-hozzájárulása, Y_i pedig a nyilvános ellenőrző része; így azonosítható a hibás fél), majd összegzi őket: $z = \sum_{i \in S} z_i$. A végeredmény a $\sigma = (R, z)$ csoport-aláírás, amely *egy közönséges Schnorr-aláírásként*

ellenőrizhető a Y nyilvános kulccsal – kívülről semmiben sem különbözik egyetlen fél által készített aláírástól.

Miért lesz $z = \sum_{i \in S} z_i$ épp egy érvényes Schnorr-aláírás? Vezessük be az $F_3(x) = F_2(x) + c \cdot F_1(x)$ polinomot, ahol F_1 a titkos kulcs s Shamir-megosztását ($Y = g^s$), F_2 pedig a (additívan megosztott) nonce k -t kódolja ($F_1(0) = s$, $F_2(0) = k = \sum_{i \in S} d_i + e_i \rho_i$); ekkor az egyes válaszok pontosan $z_i = d_i + e_i \rho_i + \lambda_i s_i c = \lambda_i F_3(i)$ alakúak¹, tehát az összegzés $z = \sum_{i \in S} \lambda_i F_3(i)$ nem más, mint az F_3 Lagrange-interpolációja a 0 pontban, azaz $z = F_3(0) = k + c \cdot s$. Mivel emellett a commitmentek szorzata épp $R = g^k$ és a kihívás $c = H_2(R \| Y \| m)$, a (R, z) pár kielégíti a standard Schnorr-egyenletet ($z = k + cs$, ellenőrzésnél $g^z = R \cdot Y^c$), ezért egy közönséges, egyfelhasználós Schnorr-aláírásként hitelesül.

Az Ábra 1 egy konkrét, négy szereplős futást szemléltet: négy egyenrangú aláíró (P_1, P_2, P_3, P_4) közösen állít elő egy aláírást. Az ábra mutatja, mely mennyiségeket számol ki, ellenőriz és küld szét az egyes fél.

2.6.4 Mit érünk el, milyen felhasználása van?

A FROST-tal egyetlen körben, párhuzamosság-korlátozás nélkül állítható elő threshold Schnorr-aláírás, amely méretében és ellenőrzésében megegyezik egy normál aláírással. Mivel az előfeldolgozás elválik az aláírástól, a felek aszinkron módon is részt vehetnek: egy részük akár offline maradhat, amíg legalább t fél elérhető. Tipikus felhasználási területek: kriptovaluta-tárcák és letétkezelők kulcsvédelme (a privát kulcsot soha nem rakják össze egyetlen helyen), elosztott hitelesítésszolgáltatók és konszenzus-hálózatok aláírásai, illetve DAO-k és intézményi többaláírás működés. A FROST-ot azóta az IRTF/CFRG szabványosította: a protokoll kétkörös változatát a **RFC 9591** [34] rögzíti. Számos éles (production) és nyílt forráskódú implementáció létezik: a séma egyik szerzőjéhez, a Zcash Foundationhoz köthető referencia Rust-könyvtár [35], valamint a DFNS *Givre* MPC-könyvtára [36], amelyet több blokklánc (Bitcoin, Solana, Stellar, Tezos) tárca- és letétkezelő-infrastruktúrája használ. A Schnorr-kompatibilitás miatt közvetlenül használható a Bitcoin Taproot (BIP 340 [22]) aláírásaival is; a Bitcoin-ökoszisztémában emiatt külön BIP-tervezet készül a Taproot-kompatibilis FROST-aláírássra [37], és a Blockstream is FROST-alapú, a láncon közönséges egyaláírás tárcaként megjelenő multisig megoldáson dolgozik.

3 BLS aláírás

2001-ben Dan Boneh, Ben Lynn és Hovav Shacham publikálták a BLS aláírási sémát [29], amely az addigi aláírási algoritmusoktól eltérően bilineáris párosítások (pairings) matematikáját használja elliptikus görbéken. A motiváció az volt, hogy a Schnorr és az ECDSA

¹Hogy pontosan miért $z_i = \lambda_i F_3(i)$, azon múlik, hogy a z_i két tagja *kétféle* titokmegosztásból származik. A titkos kulcs s Shamir-módon van megosztva ($s_i = F_1(i)$ egy polinom-pont), ezért a rekonstrukcióhoz a λ_i Lagrange-súly kell: $s = \sum_i \lambda_i s_i$. A nonce k ezzel szemben *additívan* van megosztva ($k = \sum_i (d_i + e_i \rho_i)$, súly nélkül). A kettőt úgy hozzuk közös nevezőre, hogy az F_2 polinomot épp az $F_2(i) = (d_i + e_i \rho_i)/\lambda_i$ pontokon átmenőnek definiáljuk; ekkor $\lambda_i F_2(i) = d_i + e_i \rho_i$. Mivel $F_3(i) = F_2(i) + c F_1(i) = F_2(i) + c s_i$, a λ_i -vel szorzás épp a válasz két tagját adja vissza: $\lambda_i F_3(i) = \underbrace{\lambda_i F_2(i)}_{d_i + e_i \rho_i} + \underbrace{\lambda_i c s_i}_{\lambda_i s_i c} = z_i$. Így egyetlen Lagrange-interpoláció egyszerre kezeli helyesen a Shamir-

módú kulcsot és az additív nonce-ot (ez utóbbinál a λ_i és az $1/\lambda_i$ kiejti egymást: $F_2(0) = \sum_i \lambda_i F_2(i) = \sum_i (d_i + e_i \rho_i) = k$).

aláírások – bár hatékonyak – nem teszik lehetővé az aláírások triviális aggregációját. A Schnorr aláírásoknál elméletileg lehetséges az aggregáció (pl. a MuSig protokollal [32]), de a nonce-ok koordinálása komoly nehézséget jelent: az aláíróknak interaktívan, több körben kell egyeztetniük a véletlenszerű nonce értékeket, ami bonyolult és hibalehetőségeket rejt. Az ECDSA esetén a helyzet még nehezebb: a nem-lineáris struktúra (k^{-1} szerepel az aláírásban) miatt a többrésztvevős aláírás (threshold ECDSA) jóval bonyolultabb protokollokat igényel [38]. A BLS-nél ezzel szemben egyáltalán nincs nonce – az aláírás teljesen determinisztikus –, így az aggregáció triviálissá válik: egyszerűen össze kell adni a görbepontokat, nincs szükség előzetes koordinációra. A BLS aláírás forradalmian új tulajdonsága, hogy tetszőleges számú aláírás egyetlen rövid aláírássá aggregálható, és az aggregált aláírás egyetlen ellenőrzési lépéssel igazolható. Ez a blokkláncok skálázhatósága szempontjából rendkívül értékes.

3.1 Bilineáris párosítás

A BLS aláírás megértéséhez szükségünk van a bilineáris párosítás (pairing) fogalmára. Legyen $\mathbb{G}_1$ és $\mathbb{G}_2$ két (additívan írt) elliptikus görbe csoport, $\mathbb{G}_T$ pedig egy (multiplikatívan írt) csoport, mindhárom q -rendű. Egy $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ leképezés bilineáris párosítás, ha teljesíti:

1. Bilinearitás: $\forall P \in \mathbb{G}_1, Q \in \mathbb{G}_2, a, b \in \mathbb{Z}$:

$$e(aP, bQ) = e(P, Q)^{ab}$$

2. Nem-degeneráltság: ha P generálja $\mathbb{G}_1$ -et és Q generálja $\mathbb{G}_2$ -t, akkor $e(P, Q) \neq 1$.
3. Hatékonyan számítható: a párosítás polinomiális időben kiszámítható (pl. Weil- vagy Tate-párosítás).

A bilinearitás a kulcstulajdonság: lehetővé teszi, hogy a kitevőket “áthúzzuk” a párosítás argumentumai között. Ez az, ami az aggregációt és a BLS aláírás ellenőrzését lehetővé teszi.

3.2 Weil párosítás

3.2.1 Történeti háttér

André Weil (1906–1998) francia matematikus a Weil-párosítást az 1940-es években fedezte fel [71], miközben a Riemann-hipotézis véges testek feletti változatán dolgozott. Egy híres levelében a rouen-i börtönből írta (ahol háborús szolgálatmegtagadás miatt ült): „A matematikai munkám a legvadabb reményeimet is felülmúlja, és már kicsit aggódom – ha csak börtönben dolgozom ilyen jól, vajon el kell-e intéznem, hogy minden évben két-három hónapot bezárva töltssek?” A klasszikus Riemann-hipotézis a Riemann-zéta függvény

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s} = \prod_{p \text{ prím}} \frac{1}{1 - p^{-s}}$$

nemtriviális zérushelyeinek elhelyezkedéséről szól (a sejtés szerint mind a $\text{Re}(s) = 1/2$ egyenesen vannak). A hipotézis a matematika egyik legfontosabb megoldatlan problémája (a Clay Intézet 1 millió dolláros Millennium-díjainak egyike), mert a zéta-függvény

zérushelyei közvetlenül meghatározzák a prímszámok eloszlását. Ha igaz: a prímszámláló függvény ($\pi(x)$, a x -nél kisebb prímek száma) a lehető legjobb közelítéssel becsülhető: $\pi(x) = \text{Li}(x) + O(\sqrt{x} \log x)$, azaz a prímek „szabályosan” oszlanak el, nincs bennük váratlan csomósodás vagy ritkulás. Ha hamis lenne: léteznének a prímek eloszlásában drámai szabálytalanságok, és sok, a prímek regularitására épülő tétel megdőlné. Kriptográfiai szempontból például a Miller–Rabin prímteszt hibabecslése, az RSA kulcsgenerálás során a nagy prímek sűrűségére vonatkozó feltételezések, és bizonyos hash-függvények ütközés-ellenállóságának bizonyításai épülnek (közvetve) a prímek szabályos eloszlására. Weil a véges testek feletti megfelelőjét vizsgálta: ha C egy görbe $\mathbb{F}_q$ felett, hány $\mathbb{F}_{q^n}$ -beli pontja van? A válasz a görbe zéta-függvényében kódolva van – ez egy formális hatványsor, amely az összes testbővítés feletti pontszámot egyetlen objektumba csomagolja:

$$Z(C/\mathbb{F}_q, T) = \exp \left(\sum_{n=1}^{\infty} \frac{|C(\mathbb{F}_{q^n})|}{n} T^n \right)$$

Elliptikus görbékre ez racionális függvénnyé egyszerűsödik:

$$Z(E/\mathbb{F}_q, T) = \frac{1 - aT + qT^2}{(1 - T)(1 - qT)}$$

ahol $a = q + 1 - |E(\mathbb{F}_q)|$ az ún. Frobenius nyoma. A „Riemann-hipotézis görbékre” azt mondja ki, hogy a számláló gyökei $(\alpha, \bar{\alpha})$ a $|\alpha| = \sqrt{q}$ körön fekszenek – és ebből következik a Hasse-korlát: $|N - (q + 1)| \leq 2\sqrt{q}$, ahol N a görbe $\mathbb{F}_q$ -beli pontjainak száma.

A Frobenius-endomorfizmus az $\mathbb{F}_q$ feletti elliptikus görbén a következő leképezés:

$$\phi_q : (x, y) \mapsto (x^q, y^q)$$

Ez a görbe önmagába képezése (endomorfizmus), mert a Frobenius-homomorfizmus miatt (x^q, y^q) is kielégíti a görbe egyenletét. Egy pont pontosan akkor $\mathbb{F}_q$ -beli, ha fixpontja a Frobenius-nak ($\phi_q(P) = P$, mert $x^q = x$ pontosan az $\mathbb{F}_q$ elemeire teljesül).

A Frobenius „sajátértékei” az endomorfizmus karakterisztikus polinomjának gyökei. A ϕ_q eleget tesz a következő másodfokú egyenletnek (mint endomorfizmusok egyenlősége):

$$\phi_q^2 - a\phi_q + q = 0$$

ahol $a = q + 1 - |E(\mathbb{F}_q)|$ a Frobenius nyoma. A karakterisztikus polinom gyökei $\alpha, \bar{\alpha} \in \mathbb{C}$:

$$t^2 - at + q = 0 \implies \alpha, \bar{\alpha} = \frac{a \pm \sqrt{a^2 - 4q}}{2}$$

Ezeket nevezzük a Frobenius sajátértékeinek. A Hasse-korlát ($|a| \leq 2\sqrt{q}$) ekvivalens azzal, hogy $|\alpha| = |\bar{\alpha}| = \sqrt{q}$ – vagyis a gyökök a $\sqrt{q}$ sugarú körön vannak, ami pontosan a véges testek feletti „Riemann-hipotézis”. A pontszám ebből: $|E(\mathbb{F}_q)| = q + 1 - (\alpha + \bar{\alpha})$.

Weil a párosítást azért vezette be, mert az abeli varietások Frobenius-endomorfizmusának sajátértékeit kellett meghatároznia a bizonyításhoz – és ezek a sajátértékek pontosan a zéta-függvény gyökei. A párosítás ezt a következőképpen teszi lehetővé: a Frobenius ϕ_q hat az $E[r]$ torzió-alcsoporton, de az elliptikus görbe csoportjában nehéz közvetlenül „lát-ni” a hatását. A párosítás $e_r : E[r] \times E[r] \rightarrow \mu_r$ átvezeti a problémát a μ_r (egységgyökök) csoportjába, ahol a Frobenius hatása egyszerű hatványozás: $e_r(\phi_q(P), \phi_q(Q)) = e_r(P, Q)^q$. Ebből a Frobenius sajátértékei kiolvashatók, mert ha $\phi_q(P) = \alpha P$ (sajátvektoron), akkor $e_r(\alpha P, Q) = e_r(P, Q)^\alpha$, tehát α megjelenik kitevőként a μ_r -ben. A párosítás eredetileg tisztán algebrai geometriai eszköz volt az abeli varietások vizsgálatára; a kriptográfiai alkalmazások csak az 1990-es és 2000-es években jelentek meg.

3.2.2 Matematikai alapok

A Weil-párosítás megértéséhez a következő objektumok szükségesek:

Divízorok: egy elliptikus görbén E a divízor a görbe pontjainak formális összege, pl. $D = 3(P) - 2(Q) - (O)$, ahol O a végtelenben lévő pont. A divízorok foka a koefficiensek összege ($3 - 2 - 1 = 0$ a fenti példában). A nulladfokú divízorok csoportot alkotnak.

Racionális függvények: egy f racionális függvény az E görbén egy $f : E \rightarrow \overline{\mathbb{F}}_q$ leképezés, amelynek zérushelyei és pólusai vannak. Minden ilyen f -hez tartozik egy divízor $\text{div}(f)$, amely leírja a zérushelyek és pólusok helyét és multiplicitását. Pl. ha f -nek egyszeres zérushelye van P -ben és egyszeres pólusa Q -ban, akkor $\text{div}(f) = (P) - (Q)$.

Torzió-pontok: az $E[r]$ jelöli azokat a pontokat, amelyekre $rP = O$ (az r -edrendű torzió-alcsoport). $\overline{\mathbb{F}}_q$ feletti elliptikus görbéknél, ha $\gcd(r, q) = 1$, akkor $E[r] \cong \mathbb{Z}_r \times \mathbb{Z}_r$ az algebrai lezárt felett.

3.2.3 A Weil-párosítás definíciója

A Weil-párosítás egy bilineáris leképezés:

$$e_r : E[r] \times E[r] \rightarrow \mu_r$$

ahol μ_r az r -edik egységgyökök csoportja $\overline{\mathbb{F}}_q$ -ban. A definíció megértéséhez a következő logikát kell követni:

Miért léteznek a szükséges racionális függvények? Mivel $P \in E[r]$, azaz $rP = O$, a divízor $r \cdot [(P) - (O)]$ főosztó (Abel tétele értelmében). Ez garantálja, hogy létezik egy f_P racionális függvény amelyre $\text{div}(f_P) = r(P) - r(O)$ – tehát f_P -nek r -szeres zérushelye van P -ben és r -szeres pólusa O -ban. Hasonlóan létezik f_Q is.

Miért hányados a definíció? Az $f_P(D_Q)$ kiértékelés önmagában függ a D_Q divízor konkrét választásától, tehát nem jól definiált. Viszont az

$$e_r(P, Q) = \frac{f_P(D_Q)}{f_Q(D_P)}$$

hányados már invariáns: nem függ a divízorok reprezentációjától. A szimmetrizálás (hányadosképzés) az, ami jól definiálttá teszi a párosítást. Weil meglepő eredménye, hogy ez a hányados ráadásul bilineáris és nem-degenerált is – tehát valóban hasznos algebrai eszköz.

Adott $P, Q \in E[r]$ pontokra, legyen D_P és D_Q két nulladfokú divízor, amelyekre $D_P \sim (P) - (O)$ és $D_Q \sim (Q) - (O)$ (lineárisan ekvivalensek), és $\text{supp}(D_P) \cap \text{supp}(D_Q) = \emptyset$ (diszjunkt tartók). Legyen f_P és f_Q racionális függvények, amelyekre $\text{div}(f_P) = r \cdot D_P$ és $\text{div}(f_Q) = r \cdot D_Q$. Az f_P a divízora által (konstans szorzó erejéig) egyértelműen meghatározott: ha f'_P is teljesíti $\text{div}(f'_P) = r \cdot D_P$ -t, akkor $f'_P = c \cdot f_P$ valamely $c \in \overline{\mathbb{F}}_q^*$ -ra. Ez a konstans nem befolyásolja a párosítást, mert D_Q nulladfokú ($\sum n_i = 0$), tehát $(c \cdot f_P)(D_Q) = c^{\sum n_i} \cdot f_P(D_Q) = c^0 \cdot f_P(D_Q) = f_P(D_Q)$. Így a párosítás jól definiált. Ekkor a fenti hányadossal definiáljuk $e_r(P, Q)$ -t, ahol $f(D) = \prod_i f(P_i)^{n_i}$ a divízor pontjaira kiértékelve.

A Weil-párosítás tulajdonságai és azok eredete:

- Bilinearitás: $e_r(aP, bQ) = e_r(P, Q)^{ab}$. Ez abból következik, hogy f_{aP} és f_P^a ugyanolyan divízorral rendelkeznek (mindkettőé $ra(P) - ra(O)$), tehát csak egy konstans szorzóban különbözhetnek. A kiértékelés során ez a konstans kitevőbe kerül, ami a bilinearitást adja.

- Nem-degeneráltság: ha $P \neq O$, létezik Q amelyre $e_r(P, Q) \neq 1$. Ez onnan jön, hogy $E[r] \cong \mathbb{Z}_r \times \mathbb{Z}_r$ (két független generátorral), és a racionális függvények f_P nem lehetnek triviálisak nemtriviális P -re – ha minden Q -ra 1 lenne, az azt jelentené, hogy f_P konstans a Q -k divízorain, ami ellentmondana $\text{div}(f_P) \neq 0$ -nak.
- Alternáló: $e_r(P, P) = 1$. Ez a hányadosképzés szimmetriájából következik: $f_P(D_P)/f_P(D_P) = 1$ (ha $D_P = D_Q$, a hányados triviálisan 1). A diszjunkt tartó feltétel miatt a pontos bizonyítás határátmenetet igényel, de az intuíció helyes.

3.2.4 Miller-algoritmus: implementáció

A Weil-párosítás kiszámítása a gyakorlatban a Miller-algoritmussal történik, amely egy konkrét recept az f_P racionális függvény iteratív felépítésére. Az ötlet: nem keressük meg f_P -t, hanem a double-and-add módszerrel konstruáljuk, hasonlóan a skalárszorzáshoz.

Az algoritmus az $f_{i,P}$ függvényeket építi fel lépésenként, ahol $\text{div}(f_{i,P}) = i(P) - (iP) - (i-1)(O)$. Minden lépésben a görbe pontjain átmenő egyenesek egyenleteit használjuk (ezek racionális függvények ismert zérushelyekkel és pólusokkal):

- Duplázás: $f_{2i} = f_i^2 \cdot \frac{\ell_{iP,iP}}{v_{2iP}}$, ahol $\ell_{iP,iP}$ az iP -beli érintő egyenes egyenlete ($y - \lambda x - \nu = 0$), v_{2iP} pedig a $2iP$ ponton átmenő vertikális egyenes ($x - x_{2iP} = 0$).
- Összeadás: $f_{i+1} = f_i \cdot f_1 \cdot \frac{\ell_{iP,P}}{v_{(i+1)P}}$, ahol $\ell_{iP,P}$ az iP és P összekötő egyenese.

Tehát a teljes folyamat: görbe + P pont $\xrightarrow{\text{Miller}}$ f_P függvény $\xrightarrow{\text{kiértékelés}}$ $f_P(D_Q)$ $\xrightarrow{\text{hányados}}$ $e_r(P, Q)$. A Weil-párosítás mindkét irányban futtatja a Miller-algoritmust (f_P és f_Q), majd veszi a hányadosukat. A gyakorlatban gyakran a Tate-párosítást (amelyet John Tate definiált az 1950-es években, és Frey–Rück [75] alkalmazott először a DLP redukciójára) használják helyette, amely csak egyszer futtatja a Miller-algoritmust (tehát kétszer olyan gyors), egy végső hatványozás árán.

Victor Miller 1986-ban előadta az algoritmust, de formálisan csak 2004-ben publikálta [70]. Az ok valószínűleg az volt, hogy az 1980-as években a párosításokat még kizárólag támadásra használták (MOV-algoritmus), és nem volt világos, hogy konstruktív kriptográfiai alkalmazásuk is lehet. A publikálás késedelme miatt a Miller-loop implementációs részleteit sokan újra felfedezték a közbeeső években.

3.2.5 Kriptográfiai alkalmazások

A párosítások kriptográfiai forradalma az 2000-es években kezdődött:

- MOV-algoritmus (1993) [72]: Menezes, Okamoto és Vanstone megmutatták, hogy a Weil-párosítás segítségével az elliptikus görbe DLP-je redukálható a véges test multiplikatív csoportjának DLP-jére, ahol szubexponenciális algoritmusok (index calculus) léteznek. Az algoritmus a következő: adott $P, Q \in E(\mathbb{F}_q)$ ahol $Q = nP$ és $r = \text{ord}(P)$, keresünk egy $R \in E[r]$ pontot amelyre $e_r(P, R) \neq 1$. Kiszámítjuk:

$$\alpha = e_r(P, R) \in \mathbb{F}_{q^k}^*, \quad \beta = e_r(Q, R) = e_r(nP, R) = e_r(P, R)^n = \alpha^n \in \mathbb{F}_{q^k}^*$$

A bilinearitás miatt az eredeti ECDLP ($Q = nP$ az E görbén) átalakult egy hagyományos DLP-vé ($\beta = \alpha^n$ az $\mathbb{F}_{q^k}^*$ multiplikatív csoportban). A k az ún. embedding degree: a legkisebb egész amelyre $r \mid q^k - 1$. Ha k kicsi (pl. $k \leq 6$), az index

calculus hatékonyan megoldja a DLP-t $\mathbb{F}_{q^k}^*$ -ban, tehát a görbe nem biztonságos. A kriptográfiai gyakorlatban tehát kétféle stratégia létezik: (1) ha nem akarunk párosítást használni (pl. ECDSA, Schnorr), nagy embedding degree-jű görbét választunk, hogy az MOV-támadás ne működjön; (2) ha párosítást akarunk használni (pl. BLS aláírás, IBE), szándékosan közepes k -t választunk (pl. BLS12-381: $k = 12$), hogy a párosítás hatékonyan számolható legyen, de a DLP $\mathbb{F}_{q^{12}}^*$ -ban még mindig nehéz maradjon.

- Joux háromszereplős Diffie–Hellman (2000) [73]: a klasszikus Diffie–Hellman protokoll két fél között működik, és három félre való kiterjesztése korábban legalább két kommunikációs kört igényelt. A bilineáris párosítás lehetővé teszi, hogy három fél (Alice, Bob, Carol) egyetlen körben állapodjon meg egy közös titkon. A protokoll: Alice választ a -t és közzéteszi aP -t, Bob választ b -t és közzéteszi bP -t, Carol választ c -t és közzéteszi cP -t. Ezután mindegyik fél kiszámítja a közös kulcsot:

$$K_A = e(bP, cP)^a, \quad K_B = e(aP, cP)^b, \quad K_C = e(aP, bP)^c$$

A bilinearitás miatt mindhárom egyenlő: $K_A = K_B = K_C = e(P, P)^{abc}$. Ezt az értéket egyetlen fél sem tudja kiszámítani a többiek titkos értékei nélkül.

- Identity-based encryption (Boneh–Franklin, 2001) [74]: a hagyományos nyilvános kulcsú titkosításhoz a fogadónak előbb generálnia kell egy kulcspárt, és a publikus kulcsot hitelesítenie kell (PKI, tanúsítvány). Az IBE-ben a fogadó publikus kulcsa maga az identitása (pl. e-mail cím), nincs szükség PKI-ra. A rendszer négy algoritusból áll:

- Setup: egy megbízható hatóság (Private Key Generator, PKG) választ egy titkos $s \in \mathbb{Z}_r$ -t és közzéteszi a publikus paramétereket: $P, P_{\text{pub}} = sP, H_1, H_2$ (ahol H_1 egy hash-to-point, H_2 egy hash függvény).
- Extract: a PKG kiszámítja a felhasználó privát kulcsát az identitásából: $d_{\text{ID}} = s \cdot H_1(\text{ID})$. Ezt biztonságos csatornán elküldi a felhasználónak.
- Encrypt: a küldő az ID-ből kiszámítja $Q_{\text{ID}} = H_1(\text{ID})$, választ véletlen r -t, és a titkosított szöveg:

$$C = \langle rP, M \oplus H_2(e(Q_{\text{ID}}, P_{\text{pub}})^r) \rangle$$

- Decrypt: a fogadó a privát kulcsával visszafejti:

$$M = C_2 \oplus H_2(e(d_{\text{ID}}, C_1)) = C_2 \oplus H_2(e(s \cdot Q_{\text{ID}}, rP)) = C_2 \oplus H_2(e(Q_{\text{ID}}, P_{\text{pub}})^r)$$

A bilinearitás garantálja, hogy $e(d_{\text{ID}}, rP) = e(sQ_{\text{ID}}, rP) = e(Q_{\text{ID}}, P)^{sr} = e(Q_{\text{ID}}, sP)^r = e(Q_{\text{ID}}, P_{\text{pub}})^r$, tehát a visszafejtés helyes. A rendszer hátránya a key escrow: a PKG ismeri minden felhasználó privát kulcsát. Alkalmazási területek:

- Vállalati e-mail titkosítás: a dolgozók e-mail címe a publikus kulcs, a HR osztály a PKG (és visszavonhat hozzáférést).
- IoT eszközök: az eszköz sorozatszáma vagy MAC címe a publikus kulcs, a gyártó a PKG – a titkosított firmware-frissítés az eszköz regisztrációja előtt is küldhető.

- Időzített titkosítás: a dátum mint identitás (pl. „2026-07-01”), a PKG csak azon a napon adja ki a privát kulcsot – így előre titkosítható üzenet, amely csak egy jövőbeli időpontban fejthető vissza.
- Attribútum-alapú titkosítás (ABE): az identitás helyett attribútumok halmazát használják (pl. „orvos ÉS kardiológia ÉS Semmelweis Klinika”), és csak a megfelelő attribútumokkal rendelkező felhasználó tudja visszafejtetni.
- BLS aláírás (2001): determinisztikus, aggregálható aláírás – a blokkláncok skálázhatóságának kulcsa.

3.3 BLS kulcsgenerálás

1. Adott a nyilvános paraméter: $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$, a q csoportrend, a generátorok $P \in \mathbb{G}_1$ és $Q \in \mathbb{G}_2$, a párosítás e , és egy $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$ hash függvény (amely üzeneteket görbepontokra képez). Ez utóbbi nem triviális feladat: egy tetszőleges bitsorozatból egy érvényes görbepontot kell előállítani úgy, hogy a leképezés biztonságos legyen (ne lehessen a diszkrét logaritmusát kiszámolni a kapott pontnak más ismert ponthoz képest). Ezt a problémát a hash-to-curve irodalom tárgyalja [39], a BLS12-381 görbére specifikusan Wahby és Boneh [40] mutattak hatékony, konstans idejű megoldást.
2. A titkos kulcs: véletlenszerű x , ahol $0 < x < q$.
3. A publikus kulcs: $pk = xQ \in \mathbb{G}_2$.

3.4 BLS aláírás generálás

Adott egy m üzenet:

1. Számítsuk ki a hash értéket a görbén: $h = H(m) \in \mathbb{G}_1$.
2. Az aláírás: $\sigma = x \cdot h \in \mathbb{G}_1$ (a titkos kulcs szorozva a hash ponttal).

Az aláírás egyetlen görbepont – nincs szükség nonce-ra, nincs véletlenszerű érték, az aláírás teljesen determinisztikus.

3.5 BLS aláírás ellenőrzés

Az ellenőrző ismeri $pk = xQ$, a σ aláírást, és az m üzenetet:

1. Számítsuk ki: $h = H(m)$.
2. Az aláírás érvényes, ha:

$$e(\sigma, Q) = e(h, pk)$$

Az ellenőrzés azért működik, mert a bilinearitás miatt:

$$e(\sigma, Q) = e(xh, Q) = e(h, Q)^x = e(h, xQ) = e(h, pk)$$

3.6 Aláírás aggregáció

A BLS legfontosabb tulajdonsága: ha n aláíró ($x_1, \dots, x_n$ titkos kulcsokkal) aláír n üzenetet ($m_1, \dots, m_n$), akkor az n darab aláírás ($\sigma_1, \dots, \sigma_n$) egyetlen ponttá aggregálható:

$$\sigma_{agg} = \sigma_1 + \sigma_2 + \dots + \sigma_n \in \mathbb{G}_1$$

Az aggregált aláírás ellenőrzése:

$$e(\sigma_{agg}, Q) = \prod_{i=1}^n e(H(m_i), pk_i)$$

Ez egyetlen görbepontnyi tárolóhelyet igényel az n darab külön aláírás helyett. Az Ethereum 2.0 (Beacon Chain) pontosan ezt a tulajdonságot használja ki: a validátorok (jelenleg több százezren) BLS aláírásokat generálnak, amelyeket aggregálva egyetlen aláírásként tárolnak a blokkokban [30]. A BLS12-381 görbe [31] lett az Ethereum 2.0 szabványa, amelyet kifejezetten a hatékony párosítás-számításra optimalizáltak.

3.7 BLS hátrányai

A BLS aláírás nem tökéletes. A párosítás kiszámítása lényegesen költségesebb művelet mint egy egyszerű skalárszorzás az elliptikus görbén – egy BLS ellenőrzés nagyjából 10–20-szor lassabb mint egy ECDSA vagy Schnorr ellenőrzés. Emellett a bilineáris párosítást támogató görbék (pl. BLS12-381) bonyolultabbak és kevésbé auditáltak mint a secp256k1 vagy az Ed25519. Ennek ellenére azokban a rendszerekben, ahol nagyszámú aláírást kell kezelni és az aggregáció megtakarítása meghaladja az egyedi ellenőrzés költségét, a BLS az optimális választás.

Hivatkozások

- [1] R. L. Rivest, A. Shamir, L. Adleman, *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, 21(2):120–126, 1978.
- [2] D. Coppersmith, *Finding a Small Root of a Univariate Modular Equation*, Advances in Cryptology – EUROCRYPT '96, Lecture Notes in Computer Science, vol. 1070, Springer, pp. 155–165, 1996.
- [3] W. Diffie, M. E. Hellman, *New Directions in Cryptography*, IEEE Transactions on Information Theory, 22(6):644–654, 1976.
- [4] National Bureau of Standards, *Data Encryption Standard (DES)*, FIPS PUB 46, 1977.
- [5] J. Daemen, V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer, 2001.
- [6] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018.
- [7] S. Vanstone, *Responses to NIST's Proposal*, Communications of the ACM, 35(7):50–52, 1992. (Válaszlevél, amelyben Vanstone az elliptikus görbék alkalmazását javasolja a DSA-hoz.)
- [8] American National Standards Institute (ANSI), *X9.62: Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA)*, 1998.
- [9] L. M. Adleman, C. Pomerance, R. S. Rumely, *On Distinguishing Prime Numbers from Composite Numbers*, Annals of Mathematics, 117(1):173–206, 1983.
- [10] S. Goldwasser, J. Kilian, *Almost All Primes Can Be Quickly Certified*, Proc. 18th Annual ACM Symposium on Theory of Computing, pp. 316–329, 1986.
- [11] L. M. Adleman, M.-D. Huang, *Primality Testing and Abelian Varieties Over Finite Fields*, Lecture Notes in Mathematics, vol. 1512, Springer, 1992.
- [12] M. Agrawal, N. Kayal, N. Saxena, *PRIMES is in P*, Annals of Mathematics, 160(2):781–793, 2004.
- [13] P. W. Shor, *Algorithms for Quantum Computation: Discrete Logarithms and Factoring*, Proc. 35th Annual Symposium on Foundations of Computer Science, pp. 124–134, 1994.
- [14] A. O. L. Atkin, F. Morain, *Elliptic Curves and Primality Proving*, Mathematics of Computation, 61(203):29–68, 1993.
- [15] M. O. Rabin, *Probabilistic Algorithm for Testing Primality*, Journal of Number Theory, 12(1):128–138, 1980.

- [16] J. P. Buhler, H. W. Lenstra Jr., C. Pomerance, *Factoring Integers with the Number Field Sieve*, in: A. K. Lenstra, H. W. Lenstra Jr. (eds.), *The Development of the Number Field Sieve*, Lecture Notes in Mathematics, vol. 1554, Springer, pp. 50–94, 1993.
- [17] C. P. Schnorr, *Efficient Signature Generation by Smart Cards*, Journal of Cryptology, 4(3):161–174, 1991. (Első bemutatás: CRYPTO '89, Lecture Notes in Computer Science, vol. 435, pp. 239–252, Springer, 1990.)
- [18] A. Fiat, A. Shamir, *How to Prove Yourself: Practical Solutions to Identification and Signature Problems*, Advances in Cryptology – CRYPTO '86, Lecture Notes in Computer Science, vol. 263, Springer, pp. 186–194, 1987.
- [19] M. Bellare, P. Rogaway, *Random Oracles are Practical: A Paradigm for Designing Efficient Protocols*, Proceedings of the 1st ACM Conference on Computer and Communications Security, pp. 62–73, 1993.
- [20] D. Pointcheval, J. Stern, *Security Proofs for Signature Schemes*, Advances in Cryptology – EUROCRYPT '96, Lecture Notes in Computer Science, vol. 1070, Springer, pp. 387–398, 1996.
- [21] National Institute of Standards and Technology, *Digital Signature Standard (DSS)*, FIPS PUB 186, 1994.
- [22] P. Wuille, J. Nick, T. Ruffing, *BIP 340: Schnorr Signatures for secp256k1*, Bitcoin Improvement Proposal, 2020.
- [23] L. M. Adleman, *A Subexponential Algorithm for the Discrete Logarithm Problem with Applications to Cryptography*, Proc. 20th Annual IEEE Symposium on Foundations of Computer Science, pp. 55–60, 1979.
- [24] N. Koblitz, *Hyperelliptic Cryptosystems*, Journal of Cryptology, 1(3):139–150, 1989.
- [25] J. H. Silverman, *The Arithmetic of Elliptic Curves*, 2nd edition, Graduate Texts in Mathematics, vol. 106, Springer, 2009.
- [26] L. C. Washington, *Elliptic Curves: Number Theory and Cryptography*, 2nd edition, Chapman & Hall/CRC, 2008.
- [27] Certicom Research, *SEC 2: Recommended Elliptic Curve Domain Parameters*, Standards for Efficient Cryptography Group, Version 2.0, 2010.
- [28] R. P. Gallant, R. J. Lambert, S. A. Vanstone, *Faster Point Multiplication on Elliptic Curves with Efficient Endomorphisms*, Advances in Cryptology – CRYPTO 2001, Lecture Notes in Computer Science, vol. 2139, Springer, pp. 190–200, 2001.
- [29] D. Boneh, B. Lynn, H. Shacham, *Short Signatures from the Weil Pairing*, Advances in Cryptology – ASIACRYPT 2001, Lecture Notes in Computer Science, vol. 2248, Springer, pp. 514–532, 2001.
- [30] Ethereum Foundation, *Ethereum 2.0 Specifications – Phase 0: The Beacon Chain*, 2020. <https://github.com/ethereum/consensus-specs>

- [31] S. Bowe, *BLS12-381: New zk-SNARK Elliptic Curve Construction*, Zcash Engineering Blog, 2017. A görbe a Barreto–Lynn–Scott konstrukción alapul: P. S. L. M. Barreto, B. Lynn, M. Scott, *Constructing Elliptic Curves with Prescribed Embedding Degrees*, Security in Communication Networks – SCN 2002, Lecture Notes in Computer Science, vol. 2576, Springer, pp. 263–273, 2003. A BLS12-381 paramétereinek formális specifikációja: Ethereum Foundation, *BLS12-381 Curve Specification*, EIP-2537, 2020.
- [32] Y. Lindell, *Simple Three-Round Multiparty Schnorr Signing with Full Simulatability*, Cryptology ePrint Archive, Report 2022/374, 2022.
- [33] C. Komlo, I. Goldberg, *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*, Selected Areas in Cryptography (SAC 2020), Lecture Notes in Computer Science, vol. 12804, Springer, pp. 34–65, 2021. Cryptology ePrint Archive, Report 2020/852.
- [34] D. Connolly, C. Komlo, I. Goldberg, C. A. Wood, *The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures*, RFC 9591, Internet Research Task Force (IRTF), CFRG, 2024. <https://www.rfc-editor.org/rfc/rfc9591>
- [35] Zcash Foundation, *frost: Rust implementation of FROST (Flexible Round-Optimised Schnorr Threshold signatures)*, GitHub repository. <https://github.com/ZcashFoundation/frost>
- [36] Dfns, *Givre: an open-source FROST (Flexible Round-Optimized Schnorr Threshold) MPC library*, 2024. <https://github.com/LFDT-Lockness/givre>
- [37] S. R. (siv2r) et al., *BIP-FROST Signing: FROST Signing Protocol for BIP340 Signatures*, Bitcoin Improvement Proposal draft. <https://github.com/siv2r/bip-frost-signing>
- [38] Y. Lindell, *Fast Secure Two-Party ECDSA Signing*, Advances in Cryptology – CRYPTO 2017, Lecture Notes in Computer Science, vol. 10402, Springer, pp. 613–634, 2017.
- [39] A. Faz-Hernandez, S. Scott, N. Sullivan, R. S. Wahby, C. A. Wood, *Hashing to Elliptic Curves*, Internet Research Task Force (IRTF), RFC 9380, 2023.
- [40] R. S. Wahby, D. Boneh, *Fast and Simple Constant-Time Hashing to the BLS12-381 Elliptic Curve*, Cryptology ePrint Archive, Report 2019/403, 2019.
- [41] J. von Neumann, *Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components*, in: C. E. Shannon, J. McCarthy (eds.), Automata Studies, Annals of Mathematics Studies, No. 34, Princeton University Press, pp. 43–98, 1956.
- [42] L. Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, Communications of the ACM, 21(7):558–565, 1978.
- [43] L. Lamport, *The Implementation of Reliable Distributed Multiprocess Systems*, Computer Networks, 2(2):95–114, 1978.

- [44] L. Lamport, R. Shostak, M. Pease, *The Byzantine Generals Problem*, ACM Transactions on Programming Languages and Systems, 4(3):382–401, 1982.
- [45] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008. <https://bitcoin.org/bitcoin.pdf>
- [46] V. Buterin, *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*, 2014. <https://ethereum.org/en/whitepaper/>
- [47] Y. Sompolinsky, A. Zohar, *Secure High-Rate Transaction Processing in Bitcoin*, Financial Cryptography and Data Security, Lecture Notes in Computer Science, vol. 8975, Springer, pp. 507–527, 2015.
- [48] S. Ellis, A. Juels, S. Nazarov, *ChainLink: A Decentralized Oracle Network*, 2017. <https://chain.link/whitepaper>
- [49] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, A. Bakhta, *EIP-1559: Fee Market Change for ETH 1.0 Chain*, Ethereum Improvement Proposals, 2019. <https://eips.ethereum.org/EIPS/eip-1559>
- [50] I. Eyal, E. G. Sirer, *Majority is not Enough: Bitcoin Mining is Vulnerable*, Financial Cryptography and Data Security, Lecture Notes in Computer Science, vol. 8437, Springer, pp. 436–454, 2014.
- [51] M. Mirkin, Y. Ji, J. Pang, A. Klages-Mundt, I. Eyal, A. Juels, *BDoS: Blockchain Denial-of-Service*, Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 601–619, 2020.
- [52] R. K. Konoth, E. Vineti, V. Moonsamy, M. Lindorfer, C. Kruegel, H. Bos, G. Vigna, *How You Get Shot in the Back: A Systematical Study about Cryptojacking in the Real World*, Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1701–1713, 2018.
- [53] J. Nash, *Equilibrium Points in N-Person Games*, Proceedings of the National Academy of Sciences, 36(1):48–49, 1950.
- [54] N. van Saberhagen, *CryptoNote v 2.0*, 2013. <https://www.getmonero.org/resources/research-lab/>
- [55] N. van Saberhagen, *CryptoNote v 2.0*, 2013. <https://cryptonote.org/whitepaper.pdf>
- [56] S. Noether, *Ring Confidential Transactions*, Ledger, vol. 1, pp. 1–18, 2016.
- [57] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell, *Bulletproofs: Short Proofs for Confidential Transactions and More*, IEEE Symposium on Security and Privacy, pp. 315–334, 2018.
- [58] T. Howard et al., *RandomX: ASIC-resistant Proof-of-Work Algorithm*, 2019. <https://github.com/tevador/RandomX>
- [59] R. L. Rivest, A. Shamir, Y. Tauman, *How to Leak a Secret: Theory and Applications of Ring Signatures*, Lecture Notes in Computer Science, vol. 3895, Springer, pp. 164–186, 2006.

- [60] B. Goodall, A. Noether, *Concise Linkable Ring Signatures and Forgery Against Adversarial Keys*, Monero Research Lab, 2019. <https://eprint.iacr.org/2019/654>
- [61] Internal Revenue Service, *Pilot IRS Will Pay Up to \$625,000 for Cryptocurrency Tracing Tools*, IRS Criminal Investigation, Request for Information RFI-CI-20-0006, 2020.
- [62] European Parliament and Council, *Regulation (EU) 2023/1114 on Markets in Crypto-Assets (MiCA)*, Official Journal of the European Union, L 150/52, 2023.
- [63] European Banking Authority, *Guidelines on the Transfer of Funds and Certain Crypto-Assets Regulations*, EBA/GL/2024/09, 2024.
- [64] European Parliament and Council, *Regulation (EU) 2023/1113 on Information Accompanying Transfers of Funds and Certain Crypto-Assets (Recast)*, Official Journal of the European Union, L 150/1, 2023.
- [65] A. Shamir, *How to Share a Secret*, Communications of the ACM, 22(11):612–613, 1979.
- [66] R. Canetti, N. Makriyannis, U. Peled, *Fast Secure Multiparty ECDSA with Practical Distributed Key Generation and Applications to Cryptocurrency Custody*, Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1837–1851, 2022.
- [67] Fireblocks, *MPC-CMP: Threshold ECDSA from Cryptographic Multiparty Computation*, Fireblocks Whitepaper, 2020. <https://www.fireblocks.com/what-is-mpc/>
- [68] Curv (acquired by PayPal, 2021), *Institutional Digital Asset Infrastructure*. <https://www.curv.co>
- [69] P. MacKenzie, M. K. Reiter, *Two-Party Generation of DSA Signatures*, Advances in Cryptology – CRYPTO 2001, Lecture Notes in Computer Science, vol. 2139, Springer, pp. 137–154, 2001.
- [70] V. S. Miller, *The Weil Pairing, and Its Efficient Calculation*, Journal of Cryptology, 17(4):235–261, 2004.
- [71] A. Weil, *Variétés abéliennes et courbes algébriques*, Hermann, Paris, 1948.
- [72] A. J. Menezes, T. Okamoto, S. A. Vanstone, *Reducing Elliptic Curve Logarithms to Logarithms in a Finite Field*, IEEE Transactions on Information Theory, 39(5):1639–1646, 1993.
- [73] A. Joux, *A One Round Protocol for Tripartite Diffie–Hellman*, Algorithmic Number Theory Symposium (ANTS-IV), Lecture Notes in Computer Science, vol. 1838, Springer, pp. 385–393, 2000.
- [74] D. Boneh, M. Franklin, *Identity-Based Encryption from the Weil Pairing*, Advances in Cryptology – CRYPTO 2001, Lecture Notes in Computer Science, vol. 2139, Springer, pp. 213–229, 2001.

- [75] G. Frey, H.-G. Rück, *A Remark Concerning m -Divisibility and the Discrete Logarithm in the Divisor Class Group of Curves*, Mathematics of Computation, 62(206):865–874, 1994.
- [76] R. Göbel, P. Keeler, A. E. Krzesinski, P. G. Taylor, *Bitcoin Blockchain Dynamics: The Selfish-Mine Strategy in the Presence of Propagation Delay*, Performance Evaluation, vol. 104, pp. 23–41, 2016.
- [77] A. Back, *Hashcash – A Denial of Service Counter-Measure*, 2002. <http://www.hashcash.org/papers/hashcash.pdf>
- [78] Block.one, *EOS.IO Technical White Paper v2*, 2018. <https://github.com/EOSIO/Documentation>
- [79] J. Sun, *TRON: Advanced Decentralized Blockchain Platform*, 2018. https://tron.network/static/doc/white_paper_v_2_0.pdf
- [80] Counterparty Foundation, *Counterparty: Protocol Specification*, 2014. <https://counterparty.io>
- [81] C. Yocom-Piatt, *Decred: An Autonomous Digital Currency*, 2016. <https://docs.decred.org/>
- [82] D. Mazières, *The Stellar Consensus Protocol: A Federated Model for Internet-level Consensus*, Stellar Development Foundation, 2015.
- [83] M. Castro, B. Liskov, *Practical Byzantine Fault Tolerance*, Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI), pp. 173–186, 1999.
- [84] J. Kwon, *Tendermint: Consensus without Mining*, 2014. <https://tendermint.com/static/docs/tendermint.pdf>
- [85] M. J. Fischer, N. A. Lynch, M. S. Paterson, *Impossibility of Distributed Consensus with One Faulty Process*, Journal of the ACM, 32(2):374–382, 1985.
- [86] M. E. Hellman, *A Cryptanalytic Time-Memory Trade-Off*, IEEE Transactions on Information Theory, 26(4):401–406, 1980.
- [87] H. Abusalah, J. Alwen, B. Cohen, D. Khilko, K. Pietrzak, L. Reyzin, *Beyond Hellman’s Time-Memory Trade-Offs with Applications to Proofs of Space*, Advances in Cryptology – ASIACRYPT 2017, Lecture Notes in Computer Science, vol. 10625, Springer, pp. 357–379, 2017.
- [88] D. Boneh, J. Bonneau, B. Bünz, B. Fisch, *Verifiable Delay Functions*, Advances in Cryptology – CRYPTO 2018, Lecture Notes in Computer Science, vol. 10991, Springer, pp. 757–788, 2018.
- [89] Filecoin Team, *Proof of Replication and Proof of Spacetime*, Protocol Labs Technical Report, 2017. <https://filecoin.io/proof-of-replication.pdf>
- [90] B. Cohen, K. Pietrzak, *The Chia Network Blockchain*, Chia Network, 2019. <https://www.chia.net/assets/Chia-New-Consensus-0.9.pdf>

- [91] A. Haleem, A. Allen, A. Thompson, M. Nijdam, R. Tuesta, *Helium: A Decentralized Wireless Network*, Helium Systems Inc., 2019. <http://whitepaper.helium.com>
- [92] D. Chaum, *Blind Signatures for Untraceable Payments*, Advances in Cryptology – CRYPTO '82, Springer, pp. 199–203, 1983.
- [93] C. Dwork, M. Naor, *Pricing via Processing or Combatting Junk Mail*, Advances in Cryptology – CRYPTO '92, Lecture Notes in Computer Science, vol. 740, Springer, pp. 139–147, 1993.
- [94] H. Ong, C. P. Schnorr, A. Shamir, *An Efficient Signature Scheme Based on Quadratic Equations*, Proceedings of the 16th Annual ACM Symposium on Theory of Computing, pp. 208–216, 1984.
- [95] J. M. Pollard, *The Lattice Sieve*, Lecture Notes in Mathematics, vol. 1256, Springer, pp. 43–49, 1987.
- [96] D. J. Bernstein, J. Buchmann, E. Dahmen (szerk.), *Post-Quantum Cryptography*, Springer, 2009. ISBN 978-3-540-88701-0.
- [97] L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC), pp. 212–219, 1996.
- [98] R. Babbush, A. Zalcman, C. Gidney, M. Broughton, T. Khattar, H. Neven, T. Bergamaschi, J. Drake, D. Boneh, *Securing Elliptic Curve Cryptocurrencies against Quantum Vulnerabilities: Resource Estimates and Mitigations*, arXiv:2603.28846v2, 2026.
- [99] T. Häner, F. Tripier, J. Young, M. Naehrig, A. Maksymov, S. Alam, D. Maslov, M. Parrott, Y. de Sereville, J. Sullivan, M. Webster, N. Delfosse, J. Gamble, M. Roetteler, *Computing 256-bit elliptic curve discrete logarithms in 26 days on a fault-tolerant trapped-ion quantum computer with 20,000 qubits*, arXiv:2609.05625v1 [quant-ph], 2026.
- [100] J. Proos, Ch. Zalka, *Shor's Discrete Logarithm Quantum Algorithm for Elliptic Curves*, Quantum Information & Computation, 3(4):317–344, 2003. arXiv:quant-ph/0301141.
- [101] M. Roetteler, M. Naehrig, K. M. Svore, K. Lauter, *Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms*, Advances in Cryptology – ASIACRYPT 2017, LNCS 10625, pp. 241–270, Springer, 2017.
- [102] T. Häner, S. Jaques, M. Naehrig, M. Roetteler, M. Soeken, *Improved Quantum Circuits for Elliptic Curve Discrete Logarithms*, Post-Quantum Cryptography (PQCrypto 2020), LNCS 12100, pp. 425–444, Springer, 2020.
- [103] É. Gouzien, D. Ruiz, F.-M. Le Régent, J. Guillaud, N. Sangouard, *Performance Analysis of a Repetition Cat Code Architecture: Computing 256-bit Elliptic Curve Logarithm in 9 Hours with 126 133 Cat Qubits*, Physical Review Letters, 131:040602, 2023.
- [104] D. Litinski, *How to Compute a 256-bit Elliptic Curve Private Key with Only 50 Million Toffoli Gates*, arXiv:2306.08585, 2023.

- [105] C. Chevalignard, P.-A. Fouque, A. Schrottenloher, *Reducing the Number of Qubits in Quantum Discrete Logarithms on Elliptic Curves*, Advances in Cryptology – EUROCRYPT 2026, Springer, 2026.
- [106] H. Kim, K. Jang, S. Wang, V. Srivastava, A. Baksi, G. Song, H. Seo, A. Chattopadhyay, *New Quantum Circuits for ECDLP: Breaking Prime Elliptic Curve Cryptography*, IACR ePrint 2026/106, 2026.
- [107] T. Toffoli, *Reversible Computing*, MIT Laboratory for Computer Science, Tech Memo MIT/LCS/TM-151, 1980.
- [108] S. Bravyi, A. Kitaev, *Universal Quantum Computation with Ideal Clifford Gates and Noisy Ancillas*, Physical Review A, 71:022316, 2005.
- [109] F. Arute et al., *Quantum Supremacy Using a Programmable Superconducting Processor*, Nature, 574:505–510, 2019.
- [110] IBM Quantum, *IBM Unveils 1,121-Qubit Quantum Chip ‘Condor’*, IBM Research Blog, 2023.
- [111] Y. Zhu et al., *Quantum Computational Advantage via 60-Qubit 24-Cycle Random Circuit Sampling*, Science Bulletin, 67(3):240–245, 2022.
- [112] NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, FIPS 203, 2024.
- [113] NIST, *Module-Lattice-Based Digital Signature Standard (ML-DSA)*, FIPS 204, 2024.
- [114] NIST, *Stateless Hash-Based Digital Signature Standard (SLH-DSA)*, FIPS 205, 2024.
- [115] R. J. McEliece, *A Public-Key Cryptosystem Based on Algebraic Coding Theory*, DSN Progress Report 42-44, Jet Propulsion Laboratory, pp. 114–116, 1978.
- [116] D. Coppersmith, *An Approximate Fourier Transform Useful in Quantum Factoring*, IBM Research Report RC 19642, 1994. arXiv:quant-ph/0201067.
- [117] M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2000.
- [118] J. Koch, T. M. Yu, J. Gambetta, A. A. Houck, D. I. Schuster, J. Majer, A. Blais, M. H. Devoret, S. M. Girvin, R. J. Schoelkopf, *Charge-insensitive qubit design derived from the Cooper pair box*, Physical Review A, 76(4):042319, 2007.
- [119] C. J. Ballance, T. P. Harty, N. M. Linke, M. A. Sepiol, D. M. Lucas, *High-Fidelity Quantum Logic Gates Using Trapped-Ion Hyperfine Qubits*, Physical Review Letters, 117(6):060504, 2016.
- [120] E. Knill, R. Laflamme, G. J. Milburn, *A scheme for efficient quantum computation with linear optics*, Nature, 409(6816):46–52, 2001.

- [121] D. Bluvstein, S. J. Evered, A. A. Geim, S. H. Li, H. Zhou, T. Manovitz, S. Ebadi, M. Cain, M. Kalinowski, D. Hangleiter, J. P. Bonilla Ataides, N. Maskara, I. Cong, X. Gao, P. Sales Rodriguez, T. Karolyshyn, G. Semeghini, M. J. Gullans, M. Greiner, V. Vuletić, M. D. Lukin, *Logical quantum processor based on reconfigurable atom arrays*, Nature, 626(7997):58–65, 2024.
- [122] R. B. Griffiths, C.-S. Niu, *Semiclassical Fourier Transform for Quantum Computation*, Physical Review Letters, 76(17):3228–3231, 1996.
- [123] D. Shanks, *Class Number, a Theory of Factorization, and Genera*, Proceedings of Symposia in Pure Mathematics, 20:415–440, AMS, 1971.
- [124] J. M. Pollard, *Monte Carlo Methods for Index Computation (mod p)*, Mathematics of Computation, 32(143):918–924, 1978.
- [125] V. Lyubashevsky, *Lattice Signatures Without Trapdoors*, Advances in Cryptology – EUROCRYPT 2012, LNCS 7237, pp. 738–755, Springer, 2012.